Cahier des Charges

Spécifications Techniques et Fonctionnelles

Previo

Éditeur collaboratif en ligne avec prévisualisation en temps réel (Markdown & LaTeX)

Équipe de Développement

- **Omar Chiboub**
- **Ghassen Naouar**
- **Louay Dardouri**
- **Amin Khalsi**
- **Ahmed Loubiri**
- **Ahmed Chabbouh**

Version : 1.0

Date : February 19, 2026

Type : Application Web Collaborative

Statut : Document de spécifications initiales

Contents

1 Introduction **3**

1.1 Contexte et motivation 3

1.2 Problématique 3

1.3 Objectifs du projet Previo 4

2 Présentation Générale du Système **5**

2.1 Vision globale 5

2.2 Utilisateurs cibles 5

2.3 Vue d'ensemble des cas d'utilisation 6

2.3.1 Acteurs du système 6

2.3.2 Cas d'utilisation principaux 6

3 Exigences Fonctionnelles **8**

3.1 Authentification et gestion des utilisateurs 8

3.1.1 Inscription et connexion 8

3.1.2 Gestion du profil 8

3.2 Création et gestion des documents 8

3.2.1 Création de documents 9

3.2.2 Organisation des documents 9

3.3 Choix du format d'entrée : Markdown et LaTeX 9

3.3.1 Mode Markdown 9

3.3.2 Mode LaTeX 9

3.3.3 Basculement entre les formats 10

3.4 Prévisualisation en temps réel 10

3.5 Édition collaborative en temps réel 10

3.5.1 Synchronisation des modifications 10

3.5.2 Indicateurs de présence 10

3.5.3 Contrôle des accès aux documents partagés 11

3.6 Historique des versions 11

3.7 Export des documents 11

4 Exigences Non Fonctionnelles **12**
4.1 Performance 12
4.2 Scalabilité 12
4.3 Sécurité 13
4.4 Disponibilité 13
4.5 Utilisabilité 14
5 Architecture du Système **15**
5.1 Architecture générale de haut niveau 15
5.2 Interaction entre les composants 15
5.2.1 Frontend (Next.js) 15
5.2.2 Backend (NestJS) 16
5.2.3 Services Python 16
5.3 Protocoles de communication 17
5.3.1 Conception de l'API REST 17
5.3.2 Protocole WebSocket pour la collaboration 17
6 Justification des Choix Technologiques **18**
6.1 Pourquoi Next.js pour le frontend ? 18
6.2 Pourquoi NestJS pour le backend ? 18
6.3 Pourquoi Python pour les services de rendu et de collaboration ? 19
7 Contraintes et Hypothèses **21**
7.1 Contraintes techniques 21
7.2 Contraintes de temps et de ressources 21
7.3 Hypothèses de travail 22
8 Risques et Stratégies de Mitigation **23**
9 Conclusion et Perspectives d'Évolution **25**
9.1 Conclusion 25
9.2 Perspectives d'évolution 25

Chapter 1

Introduction

1.1 Contexte et motivation

La rédaction de documents techniques et académiques constitue une activité centrale dans les environnements universitaires, scientifiques et professionnels. Des outils tels que LaTeX s'imposent depuis des décennies comme la référence en matière de composition typographique de haute qualité, tandis que Markdown s'est progressivement établi comme un format léger, lisible et polyvalent, plébiscité par les développeurs et les rédacteurs techniques.

Cependant, ces deux formats souffrent d'un écueil commun : ils nécessitent des environnements de développement locaux, souvent complexes à configurer, et ne favorisent pas la collaboration en temps réel entre plusieurs contributeurs travaillant à distance. Des solutions propriétaires comme Overleaf ont partiellement répondu à ce besoin pour LaTeX, mais aucune plateforme n'offre actuellement une solution unifiée et ouverte supportant simultanément LaTeX et Markdown dans un cadre collaboratif.

Face à l'essor du travail à distance et à la généralisation des pratiques de co-rédaction, il devient impératif de disposer d'outils permettant à plusieurs utilisateurs de travailler de manière simultanée et fluide sur un même document, quelle que soit la syntaxe qu'ils préfèrent.

1.2 Problématique

La problématique centrale à laquelle répond le projet Previo peut être formulée de la manière suivante :

> *Comment concevoir et déployer une plateforme web collaborative permettant à plusieurs utilisateurs de co-rédiger des documents en Markdown ou en LaTeX, avec une prévisualisation du rendu en temps réel, tout en garantissant la cohérence des données, la performance du système et la sécurité des échanges ?*

Les défis sous-jacents incluent :

- La gestion des conflits d'édition concurrente entre plusieurs utilisateurs ;
- Le rendu performant et fidèle de deux syntaxes distinctes (Markdown et LaTeX) ;

- La synchronisation en temps réel des modifications via des protocoles de communication adaptés ;
- La mise en œuvre d'une architecture logicielle modulaire, maintenable et évolutive ;
- La garantie de la sécurité des données et de l'authentification des utilisateurs.

1.3 Objectifs du projet Previo

Le projet Previo vise à atteindre les objectifs stratégiques et techniques suivants :

1. **Proposer un éditeur en ligne unifié** supportant à la fois le format Markdown et le format LaTeX, avec basculement possible entre les deux modes ;
2. **Offrir une prévisualisation en temps réel** du document rendu, permettant à l'utilisateur d'observer instantanément le résultat de chaque modification ;
3. **Permettre la collaboration multi-utilisateurs** en temps réel sur un même document, avec gestion des conflits et synchronisation des curseurs ;
4. **Gérer l'historique des versions** afin de permettre la restauration de n'importe quel état antérieur du document ;
5. **Fournir des fonctionnalités d'export** vers des formats portables (PDF, HTML) ;
6. **Assurer un haut niveau de sécurité** via l'authentification des utilisateurs et le contrôle d'accès aux documents ;
7. **Délivrer une expérience utilisateur fluide et intuitive**, accessible sans installation locale.

Chapter 2

Présentation Générale du Système

2.1 Vision globale

Previo est une application web collaborative de nouvelle génération, conçue pour démocratiser la rédaction de documents techniques et scientifiques. Elle se positionne comme une alternative ouverte et flexible à Overleaf, en étendant ses capacités à la prise en charge du format Markdown, et en s'appuyant sur une architecture moderne orientée microservices.

L'application repose sur trois piliers fondamentaux :

- **L'accessibilité** : aucune installation logicielle n'est requise du côté client. Un navigateur web suffit pour accéder à l'intégralité des fonctionnalités de la plateforme.
- **La collaboration** : plusieurs utilisateurs peuvent travailler simultanément sur un document partagé, avec des mécanismes robustes de résolution de conflits et de synchronisation.
- **La polyvalence** : le support natif de deux syntaxes complémentaires (LaTeX pour la précision typographique, Markdown pour la simplicité) permet d'adresser un spectre large d'utilisateurs et de cas d'usage.

2.2 Utilisateurs cibles

La plateforme Previo s'adresse aux profils suivants :

Étudiants et chercheurs
: Rédaction de mémoires, thèses, articles scientifiques et rapports de laboratoire en LaTeX ou Markdown, avec possibilité de co-rédaction avec des directeurs de recherche ou des co-auteurs.

Enseignants et formateurs
: Production de cours, de polycopiés et de supports pédagogiques collaboratifs, avec gestion centralisée des versions.

Équipes de développement logiciel
Rédaction de documentation technique en Markdown, README, wikis et spécifications fonctionnelles en environnement collaboratif.

Professionnels et consultants
Production de rapports d'expertise, de documents contractuels et de livrables techniques nécessitant un rendu de haute qualité.

Éditeurs et rédacteurs
Coordination de la rédaction multi-auteurs avec suivi des modifications et historique des révisions.

2.3 Vue d'ensemble des cas d'utilisation

Le système Previo identifie les acteurs principaux et les grands cas d'utilisation suivants :

2.3.1 Acteurs du système

- **Utilisateur non authentifié** : peut consulter la page d'accueil et s'inscrire ou se connecter à la plateforme.
- **Utilisateur authentifié** : dispose de l'ensemble des fonctionnalités de création, d'édition et de collaboration.
- **Propriétaire de document** : utilisateur ayant créé un document ; il en détient les droits d'administration (partage, suppression, gestion des accès).
- **Collaborateur** : utilisateur invité à contribuer à un document existant, avec des droits d'édition ou de lecture définis par le propriétaire.
- **Administrateur système** : responsable de la gestion de la plateforme, des utilisateurs et de la supervision des ressources.

2.3.2 Cas d'utilisation principaux

1. **Gestion du compte** : inscription, connexion, déconnexion, mise à jour du profil et réinitialisation du mot de passe.
2. **Gestion des documents** : création, duplication, archivage, suppression et organisation des documents.
3. **Édition collaborative** : modification simultanée d'un document par plusieurs utilisateurs avec synchronisation en temps réel.

4. **Prévisualisation en direct** : rendu instantané du document dans la syntaxe choisie (Markdown ou LaTeX).

5. **Gestion des versions** : consultation de l'historique des révisions et restauration d'une version antérieure.

6. **Partage et contrôle d'accès** : invitation de collaborateurs, gestion des rôles et des permissions.

7. **Export** : génération et téléchargement du document compilé au format PDF ou HTML.

Chapter 3

Exigences Fonctionnelles

3.1 Authentification et gestion des utilisateurs

3.1.1 Inscription et connexion

Le système doit permettre aux utilisateurs de créer un compte personnel sécurisé. Les fonctionnalités associées incluent :

- Inscription via un formulaire comportant : adresse électronique, nom d'affichage et mot de passe ;
- Validation de l'adresse électronique par envoi d'un courriel de confirmation ;
- Connexion par couple identifiant/mot de passe avec protection contre les attaques par force brute ;
- Support de l'authentification via des fournisseurs d'identité tiers (OAuth 2.0) : Google, GitHub ;
- Génération et gestion de jetons d'accès (JWT – JSON Web Token) avec mécanisme de rafraîchissement ;
- Déconnexion sécurisée avec invalidation du jeton côté serveur.

3.1.2 Gestion du profil

- Modification du nom d'affichage, de l'avatar et des préférences utilisateur ;
- Changement de mot de passe avec vérification de l'ancien mot de passe ;
- Réinitialisation du mot de passe via un lien envoyé à l'adresse électronique enregistrée ;
- Consultation de la liste des documents possédés et partagés.

3.2 Création et gestion des documents

3.2.1 Création de documents

- L'utilisateur authentifié peut créer un nouveau document vierge ou à partir d'un modèle prédéfini ;
- Lors de la création, l'utilisateur choisit le format d'entrée : **Markdown** ou **LaTeX** ;
- Le document est automatiquement sauvegardé sur le serveur dès sa création ;
- Chaque document est identifié par un identifiant unique (UUID) et possède un titre modifiable.

3.2.2 Organisation des documents

- L'utilisateur dispose d'un tableau de bord listant l'ensemble de ses documents (possédés et partagés) ;
- Les documents peuvent être organisés par dossiers, étiquetés, triés et filtrés ;
- Les opérations de duplication, d'archivage et de suppression permanente sont disponibles ;
- Un système de recherche plein texte sur les titres et le contenu des documents est fourni.

3.3 Choix du format d'entrée : Markdown et LaTeX

3.3.1 Mode Markdown

- Prise en charge de la syntaxe Markdown étendue (CommonMark + GFM – GitHub Flavored Markdown) ;
- Support des éléments avancés : tableaux, listes de tâches, blocs de code avec coloration syntaxique, notes de bas de page, formules mathématiques via KaTeX ;
- Rendu HTML en temps réel dans le panneau de prévisualisation.

3.3.2 Mode LaTeX

- Prise en charge d'un sous-ensemble étendu de commandes LaTeX (packages courants : `amsmath`, `graphicx`, `geometry`, `babel`, etc.) ;
- Compilation du source LaTeX en PDF par un service Python dédié (via `pdflatex` ou `lualatex`) ;
- Affichage du PDF compilé dans le panneau de prévisualisation intégré ;

- Gestion et affichage des erreurs de compilation avec localisation dans le code source.

3.3.3 Basculement entre les formats

- L'utilisateur peut choisir le format à la création du document ; une conversion assistée entre les deux formats est proposée avec avertissement sur les pertes potentielles de mise en forme.

3.4 Prévisualisation en temps réel

- L'interface est divisée en deux panneaux : éditeur de code à gauche, prévisualisation rendue à droite ;
- En mode Markdown, la prévisualisation est mise à jour de manière incrémentale à chaque frappe (délai inférieur à 200 ms) ;
- En mode LaTeX, la compilation est déclenchée automatiquement après une période d'inactivité (debouncing, délai configurable, par défaut 1,5 s) ou manuellement sur demande de l'utilisateur ;
- La position de défilement du panneau de prévisualisation est synchronisée avec celle de l'éditeur (scroll synchronization) ;
- Le panneau de prévisualisation peut être masqué ou redimensionné.

3.5 Édition collaborative en temps réel

3.5.1 Synchronisation des modifications

- Plusieurs utilisateurs peuvent éditer simultanément le même document, sans conflit destructeur, grâce à l'algorithme **CRDT** (Conflict-free Replicated Data Type) ou **OT** (Operational Transformation) ;
- Toutes les modifications sont transmises en temps réel via des connexions WebSocket persistantes ;
- La latence de propagation des modifications entre les clients connectés est inférieure à 300 ms dans des conditions réseau normales.

3.5.2 Indicateurs de présence

- La position du curseur de chaque utilisateur connecté est affichée dans l'éditeur avec un code couleur distinctif et le nom de l'utilisateur ;
- Une liste des collaborateurs actuellement actifs sur le document est visible dans la barre de navigation ;

- Un indicateur de statut signale les utilisateurs en cours de frappe.

3.5.3 Contrôle des accès aux documents partagés

- Le propriétaire peut inviter des collaborateurs via leur adresse électronique ou via un lien de partage ;
- Deux niveaux de permission sont définis : **Lecture seule** et **Édition** ;
- Les permissions sont modifiables et révocables à tout moment par le propriétaire.

3.6 Historique des versions

- Le système enregistre automatiquement des points de sauvegarde (snapshots) à intervalles réguliers et à chaque fermeture de session d'un utilisateur ;
- L'utilisateur peut accéder à la liste chronologique de toutes les révisions d'un document avec la date, l'heure et l'identité de l'auteur de chaque modification ;
- Il est possible de visualiser le contenu d'une version antérieure dans un mode lecture seule ;
- Le propriétaire peut restaurer une version antérieure comme version courante du document, avec création automatique d'un point de sauvegarde de l'état actuel avant restauration ;
- Les versions peuvent être annotées manuellement (message de commit).

3.7 Export des documents

- **Export PDF** : génération d'un fichier PDF haute qualité par le service de compilation Python ; pour les documents Markdown, une conversion en LaTeX intermédiaire ou via `pandoc` est utilisée ;
- **Export HTML** : génération d'une page HTML autonome et autosuffisante (avec styles CSS intégrés) à partir du rendu du document ;
- **Export source** : téléchargement du fichier source brut (`.md` ou `.tex`) ;
- Les exports sont générés de manière asynchrone ; une notification est envoyée à l'utilisateur lors de la disponibilité du fichier.

Chapter 4

Exigences Non Fonctionnelles

4.1 Performance

- **Temps de chargement initial** : la page principale de l'application doit être interactive (TTI – Time To Interactive) en moins de 3 secondes sur une connexion standard (10 Mbps).
- **Latence de prévisualisation Markdown** : la mise à jour du rendu doit intervenir en moins de 200 ms après la dernière frappe.
- **Délai de compilation LaTeX** : la compilation d'un document de taille standard (inférieur à 50 pages, sans images volumineuses) doit aboutir en moins de 10 secondes.
- **Synchronisation collaborative** : la propagation d'une modification entre deux clients connectés sur le même document doit être inférieure à 300 ms en conditions réseau normales.
- **Temps de réponse de l'API REST** : les requêtes courantes (authentification, récupération de la liste des documents) doivent être traitées en moins de 500 ms au 95e percentile.

4.2 Scalabilité

- L'architecture doit permettre une montée en charge horizontale de chaque composant (frontend, backend, services Python) de manière indépendante.
- Le système doit être conçu pour supporter au minimum 500 utilisateurs simultanés dans sa configuration initiale, avec une capacité d'extension à 5 000 utilisateurs concurrents via ajout de ressources.
- Le service de rendu Python doit être capable de traiter plusieurs compilations en parallèle grâce à un mécanisme de file d'attente (worker queue).
- L'utilisation de conteneurs Docker et d'un orchestrateur (Kubernetes ou équiva-

lent) est prévue pour faciliter la scalabilité.

4.3 Sécurité

- **Authentification** : les mots de passe sont stockés sous forme hachée avec l'algorithme `bcrypt` (facteur de coût ≥ 12). Les jetons JWT sont signés avec RS256.
- **Autorisation** : chaque requête API est vérifiée pour s'assurer que l'utilisateur dispose des droits nécessaires sur la ressource cible (contrôle d'accès basé sur les rôles – RBAC).
- **Communications chiffrées** : l'ensemble des échanges entre le client et le serveur s'effectue exclusivement via HTTPS (TLS 1.3) et WSS (WebSocket Secure).
- **Protection contre les injections** : le service de compilation LaTeX exécute les compilations dans des environnements sandboxés (conteneurs isolés) afin de prévenir toute injection de commandes système malveillantes.
- **Validation des entrées** : toutes les données entrantes sont validées et sanitisées côté serveur avant traitement.
- **Protection CSRF et XSS** : mise en œuvre des en-têtes HTTP de sécurité (CSP, X-Frame-Options, X-Content-Type-Options) et validation des tokens anti-CSRF.
- **Limitation de débit (Rate Limiting)** : les endpoints d'authentification et les API sensibles sont protégés contre les abus par des mécanismes de limitation du nombre de requêtes.

4.4 Disponibilité

- La plateforme vise un taux de disponibilité de **99,5%** (soit moins de 44 heures d'indisponibilité par an).
- Les mises à jour de l'application sont déployées sans interruption de service (déploiement en continu – blue/green deployment ou rolling update).
- Un système de sauvegarde automatique des données est mis en place (sauvegardes quotidiennes avec rétention de 30 jours).
- La persistance des données est assurée par une base de données avec réplication afin d'éliminer les points de défaillance uniques (SPOF).

4.5 Utilisabilité

- **Interface réactive (Responsive Design)** : l'application est utilisable sur les postes de travail (résolutions à partir de 1280×720 px) et les tablettes.
- **Accessibilité** : l'interface respecte les recommandations WCAG 2.1 niveau AA pour l'accessibilité numérique.
- **Compatibilité navigateur** : l'application est compatible avec les dernières versions des navigateurs majeurs (Chrome, Firefox, Safari, Edge).
- **Internationalisation** : l'interface est conçue pour supporter plusieurs langues (i18n), avec le français et l'anglais comme langues initiales.
- **Feedback utilisateur** : des indicateurs visuels clairs informent l'utilisateur de l'état des opérations (chargement, succès, erreur, sauvegarde en cours, etc.).
- **Documentation en ligne** : une aide contextuelle et une documentation utilisateur sont accessibles depuis l'interface.

Chapter 5

Architecture du Système

5.1 Architecture générale de haut niveau

L'architecture de Previo est fondée sur le modèle **microservices**, organisée en trois grandes couches :

1. **Couche de présentation (Frontend)** : application web monopage (SPA) développée avec Next.js, responsable de l'interface utilisateur, de l'éditeur de code et du panneau de prévisualisation.

2. **Couche métier (Backend)** : serveur d'application développé avec NestJS, exposant une API RESTful et gérant les connexions WebSocket pour la collaboration en temps réel.

3. **Couche de traitement (Services Python)** : ensemble de microservices Python responsables du rendu Markdown, de la compilation LaTeX, de la gestion des algorithmes de collaboration (CRDT/OT) et de la génération des exports.

5.2 Interaction entre les composants

5.2.1 Frontend (Next.js)

Le frontend constitue le point d'entrée unique pour l'utilisateur. Il est responsable de :

- Le rendu des pages (SSR – Server-Side Rendering pour les pages publiques, CSR pour l'application) ;
- La communication avec le backend via des requêtes HTTP (API REST) et des connexions WebSocket persistantes ;
- L'intégration de l'éditeur de code (basé sur CodeMirror 6 ou Monaco Editor) ;
- L'affichage de la prévisualisation (HTML pour Markdown, PDF.js pour LaTeX) ;
- La gestion de l'état local de l'application (avec Zustand ou Redux Toolkit).

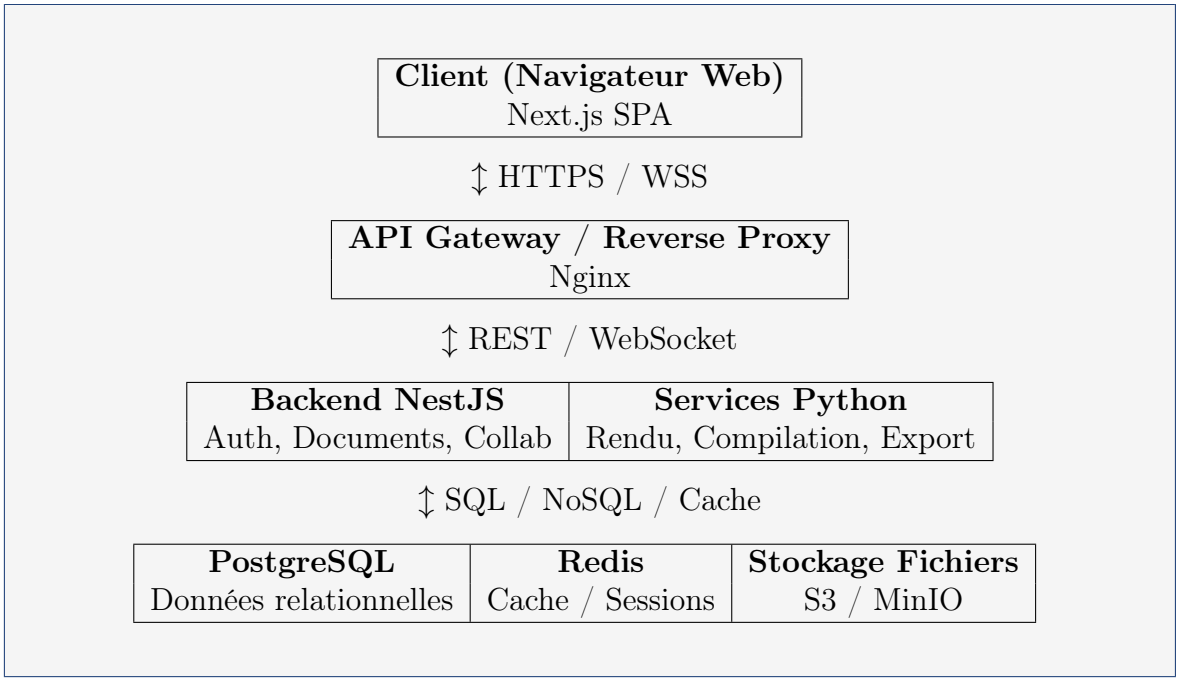

Figure 5.1: Architecture de haut niveau de la plateforme Previo

5.2.2 Backend (NestJS)

Le backend expose deux types d'interfaces :

- **API REST (HTTP/HTTPS)** : gestion de l'authentification, des utilisateurs, des documents, des permissions et des exports ;
- **Gateway WebSocket** : gestion de la collaboration en temps réel, diffusion des opérations d'édition entre clients, gestion de la présence des utilisateurs.

Le backend communique avec les services Python via des appels HTTP internes (REST) ou via un bus de messages (Redis Pub/Sub ou RabbitMQ) pour les tâches asynchrones (compilation LaTeX, génération d'exports).

5.2.3 Services Python

Les services Python sont décomposés en modules fonctionnels distincts :

- **Service de rendu Markdown** : conversion Markdown → HTML via la bibliothèque `mistune` ou `markdown-it-py`, avec support des extensions KaTeX pour les formules mathématiques ;
- **Service de compilation LaTeX** : exécution de `pdflatex` ou `lualatex` dans un conteneur Docker sandboxé, retour du PDF généré ou des messages d'erreur ;
- **Service de collaboration (CRDT)** : implémentation du moteur de transformation des opérations concurrentes, intégré via le protocole WebSocket ou via le backend NestJS ;

- **Service d'export** : orchestration de la génération des exports PDF et HTML via `pandoc` et les autres outils de rendu.

5.3 Protocoles de communication

Interaction	Description	Protocole	Format
Client ↔ Backend	Requêtes API (auth, documents)	HTTPS	JSON
Client ↔ Backend	Collaboration temps réel	WSS	JSON / Binary
Backend ↔ Python	Rendu Markdown, compilation	HTTP (interne)	JSON / bytes
Backend ↔ Python	Tâches asynchrones (export)	Redis Pub/Sub	JSON
Backend ↔ Base de données	Persistance des données	TCP	SQL (PostgreSQL)
Backend ↔ Cache	Sessions, état partagé	TCP	Redis Protocol

Table 5.1: Protocoles de communication entre les composants du système

5.3.1 Conception de l'API REST

L'API REST suit les principes RESTful et est versionnée (préfixe `/api/v1/`). Elle respecte les conventions HTTP standards pour les codes de statut et les méthodes (GET, POST, PUT, PATCH, DELETE). La documentation interactive est générée automatiquement via Swagger/OpenAPI.

5.3.2 Protocole WebSocket pour la collaboration

Les événements WebSocket échangés entre le client et le serveur suivent un protocole défini :

- `join_document` : rejoindre la session d'édition d'un document ;
- `operation` : transmettre une opération d'édition (insertion, suppression) encodée ;
- `cursor_update` : mettre à jour la position du curseur ;
- `presence_update` : informer les autres utilisateurs de l'état de présence ;
- `document_snapshot` : recevoir l'état complet du document lors de la connexion.

Chapter 6

Justification des Choix Technologiques

6.1 Pourquoi Next.js pour le frontend ?

Next.js est un framework React de premier plan qui présente plusieurs avantages décisifs pour Previo :

- **Rendu hybride (SSR/SSG/CSR)** : Next.js permet de choisir la stratégie de rendu adaptée à chaque page. Les pages publiques (page d'accueil, documentation) bénéficient du Server-Side Rendering pour un meilleur référencement (SEO) et des performances initiales optimisées. L'éditeur, en revanche, fonctionne en mode Client-Side Rendering pur pour une interactivité maximale.
- **Routage intégré** : le système de routage basé sur le système de fichiers (App Router) simplifie la gestion des routes et réduit la configuration nécessaire.
- **Optimisations automatiques** : code splitting, lazy loading, optimisation des images et des polices, réduisant la taille des bundles et améliorant les métriques de performance (Core Web Vitals).
- **Écosystème riche** : bénéficie de l'intégralité de l'écosystème React, notamment CodeMirror/Monaco Editor pour l'éditeur de code, PDF.js pour l'affichage des PDFs, et les bibliothèques de gestion d'état modernes.
- **Support TypeScript natif** : renforcement de la robustesse du code et de la maintenabilité à long terme.
- **Déploiement facilité** : compatibilité native avec les plateformes de déploiement modernes (Vercel, Docker, Kubernetes).

6.2 Pourquoi NestJS pour le backend ?

NestJS est un framework Node.js progressif qui s'impose comme le choix naturel pour le backend de Previo :

- **Architecture modulaire** : NestJS impose une architecture structurée (modules,

contrôleurs, services, fournisseurs) qui favorise la séparation des responsabilités, la testabilité et la maintenabilité du code.

- **Support natif des WebSockets** : NestJS intègre nativement le support des Gateways WebSocket via Socket.io ou `ws`, ce qui simplifie considérablement l'implémentation de la collaboration en temps réel.
- **Injection de dépendances** : le système d'injection de dépendances intégré facilite la gestion du cycle de vie des services et la mise en œuvre des tests unitaires.
- **TypeScript natif** : NestJS est écrit en TypeScript et encourage son utilisation, garantissant la cohérence avec le frontend et réduisant les erreurs de type à l'exécution.
- **Intégration ORM** : compatibilité native avec TypeORM et Prisma pour la gestion des entités de base de données et les migrations.
- **Support OpenAPI** : génération automatique de la documentation Swagger via des décorateurs, facilitant le contrat d'interface entre le frontend et le backend.
- **Guards, Interceptors et Pipes** : ces abstractions permettent d'implémenter proprement l'authentification, l'autorisation, la validation des données et la gestion des erreurs de manière transversale.

6.3 Pourquoi Python pour les services de rendu et de collaboration ?

Le choix de Python pour les services de traitement repose sur des arguments techniques solides :

- **Ecosystème de traitement de documents** : Python dispose des bibliothèques les plus matures pour le traitement de documents : `pandoc` (via `pypandoc`) pour les conversions de formats, `mistune`/`markdown-it-py` pour le rendu Markdown, `Pygments` pour la coloration syntaxique.
- **Interaction avec LaTeX** : Python permet d'orchestrer facilement les compilateurs LaTeX (`pdflatex`, `lualatex`, `xelatex`) via des appels de sous-processus et de parser leurs sorties pour extraire les erreurs.
- **Gestion asynchrone** : le framework `FastAPI` ou `aiohttp` permet de créer des API haute performance et asynchrones, idéales pour les tâches de rendu potentiellement longues.

- **Algorithmes CRDT** : des bibliothèques comme `pycrdt` offrent des implémentations éprouvées des types de données CRDT (notamment basées sur Yjs/Y.py) pour la résolution des conflits d'édition concurrente.

- **Isolation et sécurité** : l'exécution de compilations LaTeX en Python dans des conteneurs Docker permet une isolation sécurisée, limitant les risques d'injection de code malveillant.

- **Scalabilité par les workers** : l'utilisation de `Celery` avec un broker Redis ou RabbitMQ permet de distribuer facilement les tâches de compilation sur plusieurs workers, assurant la scalabilité du service.

Chapter 7

Contraintes et Hypothèses

7.1 Contraintes techniques

- **Environnement d'exécution** : l'application est conçue pour être déployée dans un environnement conteneurisé (Docker). Le déploiement sur des environnements bare-metal n'est pas officiellement supporté dans la version initiale.

- **Distribution LaTeX** : le service de compilation nécessite l'installation d'une distribution LaTeX complète (TeX Live) dans le conteneur dédié, ce qui représente un volume de données conséquent (plusieurs gigaoctets).

- **Dépendance à des services externes** : l'authentification OAuth (Google, GitHub) requiert la disponibilité de services tiers. Une panne de ces services peut impacter la fonctionnalité de connexion sociale, sans affecter l'authentification locale.

- **Limitations du rendu LaTeX en ligne** : certaines commandes LaTeX complexes (interaction avec le système de fichiers, exécution de code externe) sont délibérément désactivées pour des raisons de sécurité.

- **Taille maximale des documents** : un document source ne peut pas dépasser 10 Mo pour le mode LaTeX (hors ressources externes comme les images) afin de garantir des temps de compilation raisonnables.

- **Nombre maximal de collaborateurs simultanés** : dans la version initiale, le nombre de collaborateurs simultanés par document est limité à 20 afin de garantir la qualité de l'expérience.

7.2 Contraintes de temps et de ressources

- **Équipe** : le projet est développé par une équipe de 6 personnes, sans ressources externes dédiées.

- **Calendrier** : le développement est réalisé dans le cadre d'un projet académique ou professionnel avec une durée définie, impliquant une priorisation rigoureuse des

fonctionnalités (approche MVP – Minimum Viable Product).

- **Infrastructure initiale** : le déploiement initial s'effectue sur une infrastructure cloud de taille modeste. La montée en charge ne sera envisagée qu'après validation du MVP.
- **Budget** : en l'absence de budget alloué pour des licences logicielles, l'ensemble de la pile technologique s'appuie exclusivement sur des outils open source.

7.3 Hypothèses de travail

- Les utilisateurs disposent d'une connexion Internet stable et d'un navigateur web moderne.
- Les documents produits par les utilisateurs respectent des bonnes pratiques de rédaction LaTeX ; les documents malformés seront rejetés avec un message d'erreur explicite.
- La langue principale de l'interface pour la version initiale est le français, avec une internationalisation planifiée pour les versions ultérieures.
- Les données des utilisateurs sont stockées exclusivement dans des datacenters européens, conformément aux exigences du RGPD.

Chapter 8

Risques et Stratégies de Mitigation

Les risques identifiés dans le projet Previo et les stratégies de mitigation associées sont présentés dans le tableau ci-dessous.

Risque	**Proba**	**Stratégie de mitigation**
Complexité de l'implémentation CRDT pour la collaboration en temps réel	Élevée	Utilisation de bibliothèques éprouvées (Yjs, pycrdt). Prévoir une phase de prototypage dédiée. En dernier recours, repli sur un mécanisme de verrouillage optimiste (locking).
Performances insuffisantes du service de compilation LaTeX	Moyenne	Mise en œuvre d'un cache de compilation basé sur le hash du document source. Déploiement de plusieurs workers parallèles. Limitation de la taille des documents en entrée.
Failles de sécurité dans l'exécution de code LaTeX utilisateur	Moyenne	Isolation stricte dans des conteneurs Docker avec profils seccomp et apparmor. Liste blanche des commandes et packages LaTeX autorisés. Revue de sécurité dédiée.
Conflits de données lors de la collaboration intensive	Moyenne	Conception rigoureuse de l'algorithme CRDT. Tests de charge simulant de nombreux conflits simultanés. Mécanisme de réconciliation côté serveur.
Indisponibilité de services tiers (OAuth, CDN)	Faible	Authentification locale en secours. Servir les assets statiques depuis l'infrastructure propre au projet en cas d'indisponibilité CDN.
Dérive du calendrier liée à la complexité technique	Élevée	Approche MVP avec priorisation stricte des fonctionnalités (MoSCoW). Réunions hebdomadaires de suivi. Découpage en sprints de deux semaines.
Perte de données lors de pannes serveur	Faible	Sauvegardes automatiques régulières (toutes les 6 heures). Réplication de la base de données. Mécanisme de récupération depuis le cache local du navigateur.
Non-conformité RGPD	Faible	Analyse juridique préalable. Mise en œuvre des droits des utilisateurs (accès, rectification, suppression). Désignation d'un responsable de la protection des données au sein de l'équipe.

Table 8.1: Matrice des risques et stratégies de mitigation du projet Previo

Chapter 9

Conclusion et Perspectives d'Évolution

9.1 Conclusion

Le présent cahier des charges définit de manière exhaustive les spécifications fonctionnelles et non fonctionnelles de la plateforme Previo. Ce document constitue le contrat de base entre les membres de l'équipe de développement et l'ensemble des parties prenantes du projet.

Previo ambitionne de combler un vide dans l'écosystème des éditeurs collaboratifs en ligne en proposant une solution unifiée et ouverte pour la rédaction en Markdown et en LaTeX. En s'appuyant sur une architecture microservices moderne – Next.js pour le frontend, NestJS pour le backend, et des services Python spécialisés pour le rendu et la collaboration –, Previo est conçu pour être à la fois performant, sécurisé, scalable et agréable à utiliser.

L'équipe de développement s'engage à respecter les exigences définies dans ce document, en adoptant une méthodologie agile permettant d'adapter les priorités en fonction des retours utilisateurs et des contraintes techniques rencontrées en cours de développement.

9.2 Perspectives d'évolution

Plusieurs axes d'amélioration sont identifiés pour les versions futures de la plateforme :

- **Support de formats additionnels** : intégration de formats tels que AsciiDoc, reStructuredText (RST), et Typst pour répondre aux besoins d'une communauté plus large.

- **Éditeur visuel (WYSIWYG)** : développement d'un mode d'édition WYSIWYG pour les utilisateurs moins à l'aise avec les syntaxes de balisage, tout en conservant la synchronisation avec le code source.

- **Intégration de systèmes de contrôle de version** : connexion native à des dépôts Git (GitHub, GitLab, Gitea) pour synchroniser les documents avec des dépôts existants.

- **Intelligence artificielle** : intégration d'assistants IA pour la suggestion de contenu, la correction grammaticale et orthographique en temps réel, la génération de code LaTeX à partir de descriptions en langage naturel, et le résumé de documents.
- **Application mobile** : développement d'applications mobiles natives (iOS, Android) ou via React Native permettant la lecture et l'édition légère de documents en mobilité.
- **Commentaires et annotations** : système de commentaires en ligne permettant aux collaborateurs de laisser des annotations sur des portions spécifiques du document.
- **Tableaux de bord analytiques** : visualisation des statistiques d'utilisation, de l'activité des collaborateurs et de l'historique des contributions par utilisateur.
- **API publique** : exposition d'une API publique documentée permettant à des tiers de développer des intégrations et des extensions pour la plateforme Previo.
- **Mode hors ligne** : support de l'édition hors connexion avec synchronisation différée lors du rétablissement de la connexion, via les technologies PWA (Progressive Web App) et les Service Workers.